

Práctica (1): PROGRAMACIÓN EN RADIO DEFINIDA POR SOFTWARE (GNURADIO)

Santiago Alexander Bravo Rodriguez - 2224734

Christian Felipe Barbosa Pinto - 2212911

https://github.com/santiagoalexanderbravo1234-bot/2026_2_CommII_A1_GA1K/tree/Practica_1

Escuela de Ingenierías Eléctrica, Electrónica y de Telecomunicaciones

Universidad Industrial de Santander

2 de septiembre de 2026

Abstract—Abstract— This laboratory practice focused on the design, implementation, and evaluation of user-defined signal processing blocks within the GNU Radio Companion environment. The primary goal was to move beyond the use of predefined libraries by developing three synchronous Python-based components: an accumulator, a discrete differentiator, and a statistical analyzer. Test signals were introduced to assess and validate the logical and mathematical behavior of each block. During the implementation process, several issues in the original code were detected and corrected, including a discontinuity problem in the accumulator caused by the loss of state between consecutive sample batches. The experimental results demonstrated that the modified algorithms closely matched their theoretical behavior, confirming their reliability. Overall, the practice highlighted the flexibility of Software Defined Radio (SDR) for developing customized signal-processing solutions, particularly for metric analysis and noise reduction in real-world communication systems.

I Introducción

La Radio Definida por Software (SDR, por sus siglas en inglés) representa un cambio de paradigma en la ingeniería de telecomunicaciones, al reemplazar los componentes de hardware analógico tradicionales por procesamiento digital de señales ejecutado mediante software [1]. Esta arquitectura permite reconfigurar dinámicamente los parámetros de transmisión y recepción sin necesidad de alterar la infraestructura física del sistema [2].

En este ámbito, GNU Radio se ha consolidado como el entorno de desarrollo de código abierto predominante para la simulación, diseño e implementación de sistemas SDR [3]. Mediante una estructura modular basada en bloques de procesamiento optimizados en C++ y orquestados mediante scripts de Python, GNU Radio facilita la manipulación de señales en tiempo real e interacciona directamente con hardware de radiofrecuencia especializado, como las plataformas USRP

[4]. La programación en este marco abarca desde el diseño de filtros y moduladores fundamentales hasta el despliegue de protocolos de comunicación de alta complejidad, constituyendo una herramienta indispensable tanto en la formación académica como en la investigación industrial [5].

II Metodología

El desarrollo del laboratorio se estructuró en dos fases complementarias: la gestión del entorno colaborativo y la codificación algorítmica. Para la primera fase, se empleó el sistema de control de versiones Git, mediante el cual se crearon ramas de desarrollo individuales a partir de una rama principal. Todo el código se administró de manera local y se sincronizó en la nube para consolidar el trabajo grupal. Para la segunda fase, la manipulación de las señales requirió la creación de bloques síncronos implementados en el lenguaje de programación Python. Dentro del método `work()`, se extrajeron los vectores de datos utilizando los arreglos unidimensionales `input_items` y `output_items`. Se programaron tres bloques principales: un acumulador, un diferenciador y un tercer bloque que resumía características estadísticas de la señal, tales como el promedio, la media cuadrática, el valor RMS, la potencia promedio y la desviación estándar. A partir de esta estructura, se ensamblaron los diagramas de bloques en la interfaz gráfica y se ejecutaron las simulaciones correspondientes. Las figuras obtenidas a partir de estas simulaciones permitieron comprobar que los valores estadísticos coincidían con la base teórica matemática de las señales inyectadas.

III. RESULTADOS

Los resultados que evidenciamos en las simulaciones fueron los siguientes. Con el fin de implementar la lógica del sistema en la plataforma GNU Radio, se desa-



rollaron bloques personalizados en Python (*Embedded Python Blocks*). Esta metodología permitió codificar los algoritmos necesarios para la ejecución de los siguientes módulos:

- **Módulo Acumulador:** Implementa la sumatoria continua de la secuencia discreta de entrada $x[k]$, una operación clave para los procesos de reconstrucción y recuperación de la señal:

$$y[n] = \sum_{k=0}^n x[k] \quad (1)$$

- **Módulo Diferenciador:** Algoritmo orientado a resaltar las transiciones rápidas y variaciones de amplitud mediante el cálculo de la diferencia directa entre muestras consecutivas:

$$y[n] = x[n] - x[n-1] \quad (2)$$

- **Analizador Estadístico:** Bloque diseñado para caracterizar la señal en tiempo real mediante la extracción de métricas fundamentales, tales como la media (μ_x), la desviación estándar (σ_x), el valor eficaz (RMS) y la potencia promedio (P).

III-A. Acumulador

Para comprobar el funcionamiento del bloque acumulador, se implementó el montaje mostrado en la Figura 1. Como señal de entrada se utilizó una onda cuadrada acondicionada previamente, con valores que variaban de forma simétrica entre -1 y 1 .

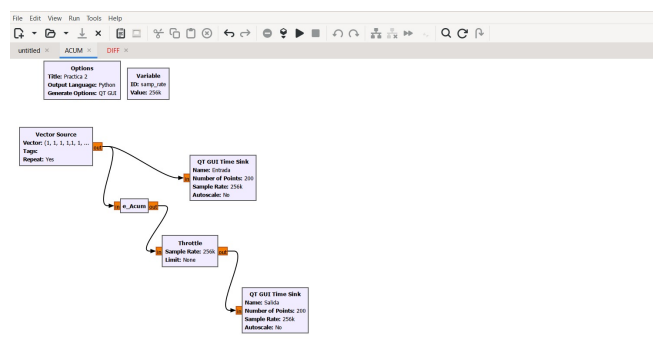


Figura 1: Diagrama de bloques del acumulador.

En la simulación del bloque acumulador se evidenció que el código inicial reiniciaba el valor de la suma cada vez que GNU Radio procesaba un nuevo bloque de muestras. Esto ocurría porque la plataforma ejecuta la función `work` varias veces de manera independiente, generando saltos o discontinuidades en la señal de salida. Para corregir este comportamiento, se agregó

una variable que almacena el último valor acumulado, permitiendo mantener una integración continua entre un bloque de datos y el siguiente.

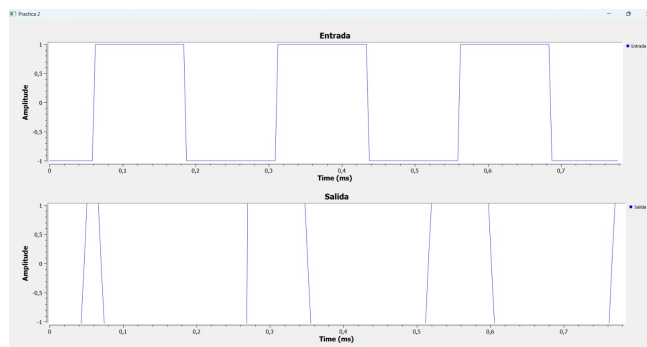


Figura 2: Simulación del funcionamiento del bloque acumulador.

La Figura 2 muestra cómo el sistema procesó la señal cuadrada para producir una salida triangular. Este hallazgo confirma que el algoritmo corregido funciona de manera óptima, ya que demuestra matemáticamente la integración continua de una señal constante por secciones, logrando mantener una pendiente estable sin caídas repentinas.

III-B Diferenciador

Con el fin de evaluar este bloque, se implementó la estructura mostrada en la Figura 3. En esta prueba, el generador se ajustó para emitir una onda triangular, acondicionada de forma idéntica al procedimiento anterior para garantizar que su amplitud oscilara entre -1 y 1.

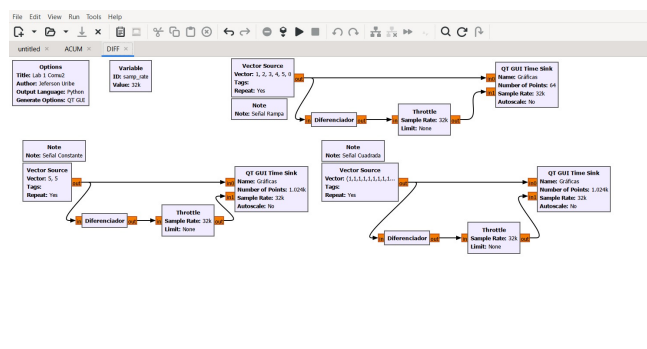


Figura 3: Diagrama de bloques del diferenciador

Si bien el código previo calculaba la resta de acumulados, la salida mantenía una dependencia con la suma total de muestras anteriores, operando como una

integración desplazada y no como un diferenciador. Por ello, se implementó la diferencia directa entre muestras consecutivas para representar con precisión la derivada discreta [2]. Tras ejecutar el sistema con esta corrección, se registraron las señales presentadas en la Figura 4.

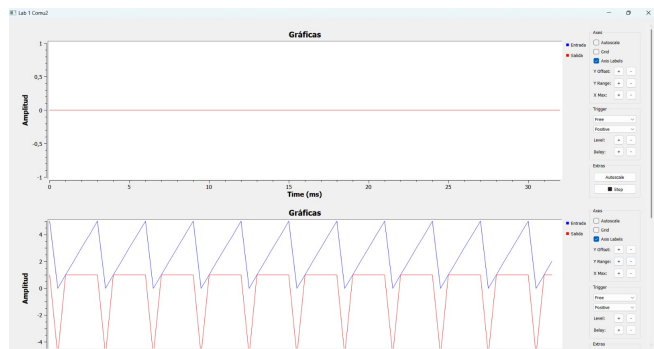


Figura 4: Simulación del funcionamiento del bloque diferenciador

La Figura 4 evidencia que, al procesar la onda triangular mediante el bloque eDiff, se obtuvo a la salida una onda cuadrada en perfecta sincronía. Dichas señales validan el correcto desempeño del algoritmo implementado. Fundamentado en que la derivada de una rampa es un valor constante, el bloque transformó la señal de entrada en los niveles alternos que caracterizan a la onda cuadrada.

III-C. Analizador Estadístico de Señales

El esquema de la Figura 5 se diseñó para evaluar el comportamiento del bloque, utilizando una fuente de vectores (Vector Source) para introducir secuencias de datos repetitivas y previamente definidas.

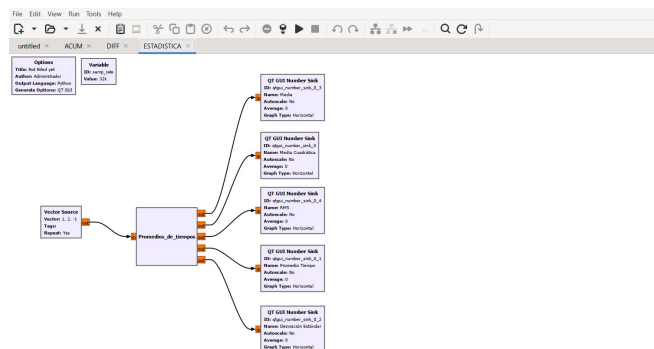


Figura 5: Diagrama de bloques del analizador estadístico de señales

A diferencia de las etapas anteriores, el código Python proporcionado en la guía base se mantuvo sin cambios

respecto a su planteamiento inicial. Para la visualización de cada parámetro de salida, el sistema se conectó a bloques QT GUI Number Sink.

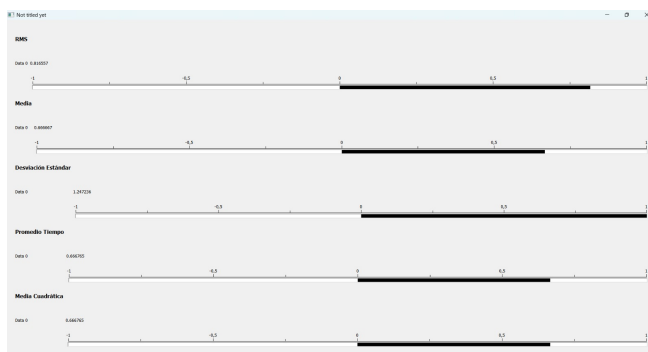


Figura 6: Interfaz gráfica con los valores estadísticos calculados para el vector (1, 2, -1)

En la Figura 6 se presentan las salidas del sistema ante el vector de prueba (1, 2, -1), ratificando la concordancia entre los datos calculados por el bloque y las estimaciones teóricas. Cabe resaltar la equivalencia observada entre la potencia promedio y el valor medio cuadrático, lo que valida matemáticamente el correcto funcionamiento del algoritmo. Finalmente, con el propósito de evaluar el desempeño del sistema ante condiciones de perturbación y sintetizar la respuesta en distintos escenarios, se contrastaron los parámetros teóricos y experimentales para diferentes niveles de ruido, cuyos datos se compilan en la Tabla 1

Vector	Núm. de muestras	Frec. de muestreo [kHz]	RMS	Desviación Estándar	Media	Media cuadrática	Potencia Promedio
(1, 2, -1)	3	32	1.4142	1.2472	0.6667	2.0000	2.0000
(2, -2, 4, -4)	4	16	3.1623	3.1623	0.0000	10.0000	10.0000
(1, -1, 2, -2, 3, -3)	6	32	2.1602	2.1602	0.0000	4.6667	4.6667
(5, 0, -5, 0, 5)	5	8	3.8730	3.7417	1.0000	15.0000	15.0000
(3, 1, 4, 1, 5, -9, 2, 6)	8	32	4.6503	4.3571	1.6250	21.6250	21.6250
(-2, -2, -2, 4, 4, 4)	6	4	3.1623	3.0000	1.0000	10.0000	10.0000

Tabla 1: Parámetros estadísticos obtenidos para diversas secuencias vectoriales.

II-A Aplicaciones

Los bloques desarrollados en esta práctica no son ejercicios aislados de programación, sino componentes fundamentales que se emplean de forma recurrente en sistemas reales de procesamiento digital de señales y comunicaciones.

El bloque *acumulador*, al comportarse como un integrador discreto, resulta esencial en procesos de filtrado paso bajo y en la demodulación de señales moduladas en frecuencia o fase, donde es necesario reconstruir la envolvente de la señal a partir de sus variaciones

instantáneas. Asimismo, este tipo de bloque tiene aplicación directa en sistemas de medición de energía y en la acumulación de lecturas provenientes de sensores industriales, donde se requiere mantener una suma continua sin pérdida de información entre ciclos de muestreo.

Por su parte, el bloque *diferenciador* permite detectar cambios abruptos en una señal, lo cual es determinante en la identificación de transiciones binarias durante la recuperación de información digital en receptores de comunicaciones. A nivel industrial, este mismo principio se aplica en el mantenimiento predictivo, donde variaciones súbitas en variables físicas como temperatura, presión o vibración pueden anticipar fallas en equipos antes de que estas se manifiesten completamente.

Finalmente, el *analizador estadístico* aporta herramientas de caracterización del canal de comunicación, dado que parámetros como la potencia promedio, el valor RMS y la desviación estándar permiten cuantificar el nivel de ruido presente en una señal y evaluar la calidad del enlace. Esta capacidad es clave en sistemas de monitoreo automático y control de calidad de señal, donde se requiere una respuesta rápida y confiable ante condiciones cambiantes del canal.

En conjunto, la posibilidad de programar estos bloques de forma personalizada (Out-of-Tree) amplía significativamente el alcance de la Radio Definida por Software, permitiendo adaptar el procesamiento de señales a necesidades específicas que las librerías predeterminadas de GNU Radio no siempre cubren.

III Conclusiones

- El desarrollo de bloques personalizados Out-of-Tree (OOT) mediante Python brindó un control preciso sobre el manejo de variables a nivel de código y mejoró la exactitud de los parámetros calculados. Esta metodología otorga mayor flexibilidad para escalar el sistema hacia aplicaciones más complejas, como la estimación de ruido en canales de comunicaciones reales.
- Los datos sintetizados en la Tabla ?? demuestran que la variación en la frecuencia de muestreo no altera el cálculo de métricas fundamentales como la potencia promedio y el valor RMS. Al procesar secuencias idénticas bajo distintas tasas de adquisición, las estimaciones estadísticas permanecieron invariantes, evidenciando la robustez del analizador implementado ante diferentes velocidades de muestreo.
- La inspección visual de las señales procesadas permitió corroborar de forma práctica los principios teóricos de la integración y derivación discretas.

Específicamente, se verificó cómo el bloque acumulador transforma una señal cuadrada en una onda triangular, mientras que el bloque diferenciador efectúa la operación inversa, convirtiendo una entrada triangular en una salida cuadrada.

Completar

Referencias

- [1] J. Mitola, "The software radio architecture," *IEEE Communications Magazine*, vol. 33, no. 5, pp. 26–38, May 1995.
- [2] J. H. Reed, *Software Radio: A Modern Approach to Radio Engineering*, Upper Saddle River, NJ, USA: Prentice Hall, 2002.
- [3] E. Blossom, "GNU Radio: Tools for exploring the radio frequency spectrum," *Linux Journal*, vol. 2004, no. 122, p. 4, Jun. 2004.
- [4] A. M. Wyglinski, D. P. Pu, M. A. Lappank, and R. T. Oetting, *Software-Defined Radio for Engineers*, Norwood, MA, USA: Artech House, 2018.
- [5] T. W. Rondeau, "GNU Radio: The free and open software radio ecosystem," *IEEE Communications Magazine*, vol. 51, no. 10, pp. 164–170, Oct. 2013.